

Hands-on 4

Difference between JPA, Hibernate and Spring Data JPA

Submitted by:

Name: Shivalika Bangaru

Course: Spring Data JPA with Spring Boot, Hibernate

College: VIT-AP University

HANDS-ON 4

Difference Between JPA, Hibernate and Spring Data JPA

Objective:

The objective of this hands-on is to understand the relationship between JPA, Hibernate, and Spring Data JPA, and to learn how Spring Data JPA simplifies database operations by reducing the amount of code required compared to Hibernate.

Introduction:

Modern Java applications often need to store and retrieve information from databases. Writing SQL queries manually for every operation can make the application lengthy and difficult to maintain. To simplify database interaction, Java provides technologies like **JPA**, **Hibernate**, and **Spring Data JPA**.

Although these technologies work together, they serve different purposes. Understanding the difference between them helps developers choose the right approach while developing enterprise applications.

What is JPA:

Java Persistence API (JPA) is a specification provided by Java for managing data between Java applications and relational databases.

JPA itself is **not a framework** and **does not contain any implementation**. Instead, it defines a set of rules and interfaces that ORM frameworks must follow.

For example, annotations such as `@Entity`, `@Table`, `@Id`, and `@Column` are part of the JPA specification.

One simple way to understand JPA is to think of it as a blueprint. It explains **what needs to be done**, but it does not explain **how it should be done**.

What is Hibernate:

Hibernate is one of the most popular **ORM (Object Relational Mapping)** frameworks used in Java.

It provides the actual implementation of the JPA specification.

Instead of writing SQL queries manually, Hibernate converts Java objects into SQL statements automatically. This reduces development time and improves code readability.

For example, saving an employee object using Hibernate automatically generates an SQL INSERT statement without requiring the developer to write SQL manually.

What is Spring Data JPA:

Spring Data JPA is a Spring Framework module that further simplifies database programming.

It does **not** implement JPA. Instead, it works on top of a JPA implementation such as Hibernate.

Spring Data JPA removes much of the repetitive code required in Hibernate. Developers only need to create a repository interface, and common database operations like saving, updating, deleting, and retrieving records are automatically available.

This makes application development faster and cleaner.

Relationship between JPA, Hibernate and Spring Data JPA:

The relationship between these technologies can be understood as follows:

Explanation:

- JPA defines the standard.
- Hibernate implements that standard.
- Spring Data JPA makes Hibernate easier to use by reducing boilerplate code.

Comparison between JPA, Hibernate and Spring Data JPA:

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Spring Module
Implementation	No	Yes	No
Developed By	Java/Jakarta EE	Hibernate Team	Spring
Database Operations	Defines rules	Performs operations	Simplifies operations
SQL Handling	No	Automatic	Automatic
Boilerplate Code	High	Medium	Very Low
Transaction Management	Not provided	Manual	Automatic using @Transactional

Hibernate Code Explanation:

In Hibernate, developers must manually perform several database-related operations.

CODE:

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
session.save(employee);
tx.commit();
session.close();
```

Explanation

- `openSession()` creates a database session.
- `beginTransaction()` starts the database transaction.
- `save(employee)` inserts the employee object into the database.
- `commit()` permanently saves the changes.
- `close()` closes the database connection.

Although Hibernate reduces SQL coding, developers still need to write transaction management code manually.

Spring Data JPA Code Explanation:

Repository

CODE:

```
public interface EmployeeRepository extends JpaRepository<Employee,
Integer> {

}
```

Service

CODE:

```
@Autowired
```

```
private EmployeeRepository employeeRepository;
```

```
@Transactional
```

```
public void addEmployee(Employee employee) {  
    employeeRepository.save(employee);  
}
```

Explanation

Spring Data JPA automatically handles session creation, transaction management, and SQL generation.

The developer only needs to call:

CODE:

```
employeeRepository.save(employee);
```

Internally, Spring Data JPA uses Hibernate to perform the database operation.

This makes the code shorter, easier to understand, and easier to maintain.

Practical Demonstration:

In the previous hands-on (**Spring Data JPA Quick Example**), a Spring Boot application was developed using Spring Data JPA and Hibernate.

The application retrieved records from the **country** table using the following method:

CODE:

```
@Transactional
```

```
public List<Country> getAllCountries() {  
    return countryRepository.findAll();  
}
```

The output displayed:

Country [code=IN, name=India]

Country [code=US, name=United States of America]

This demonstrates that Spring Data JPA internally communicates with Hibernate to execute SQL queries and fetch data from the MySQL datab

Advantages of Spring Data JPA:

- Reduces boilerplate code.
- Simplifies CRUD operations.
- Automatically manages transactions.
- Easy integration with Spring Boot.
- Improves code readability and maintainability.
- Reduces development time.

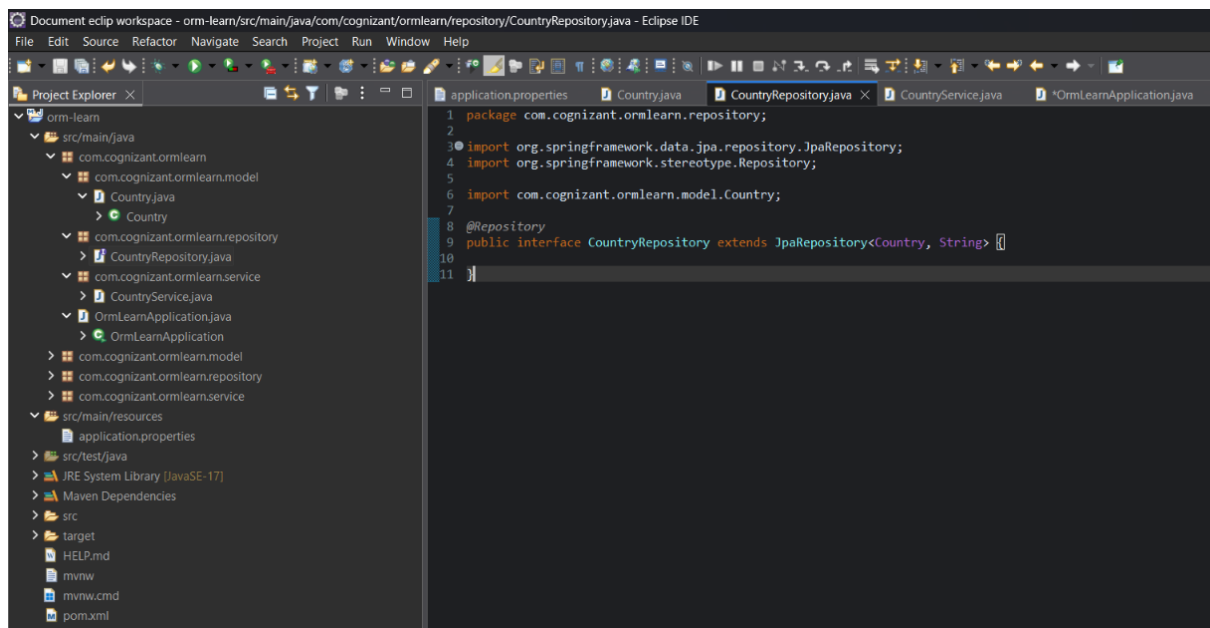
Conclusion:

JPA, Hibernate, and Spring Data JPA work together but serve different purposes. JPA provides the standard specification, Hibernate implements that specification, and Spring Data JPA simplifies the use of Hibernate by reducing repetitive coding.

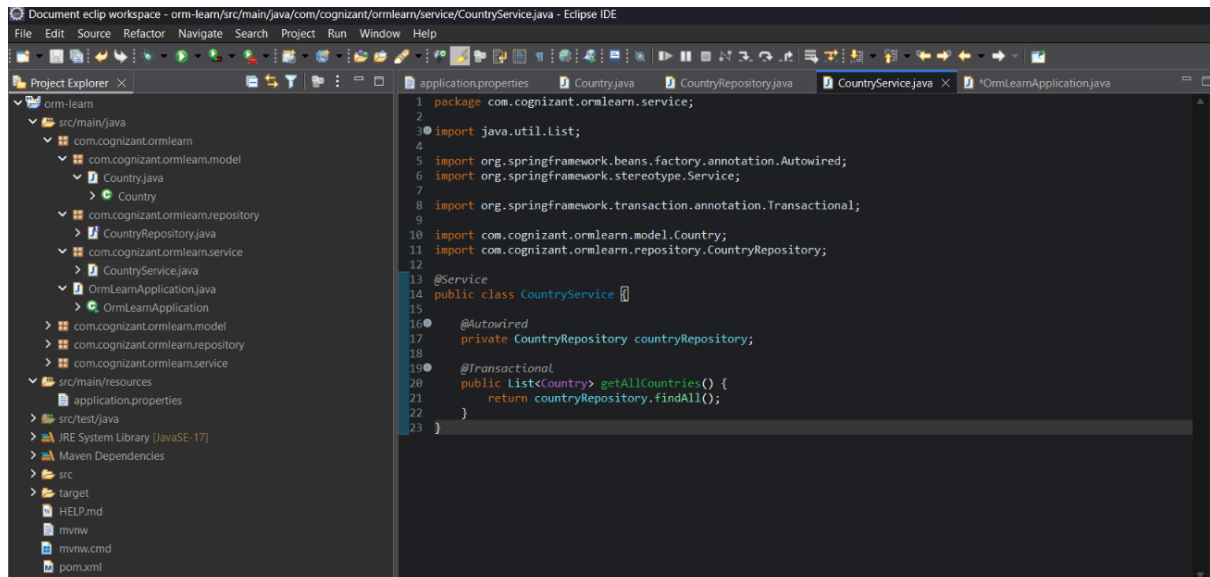
Based on the practical implementation completed in the previous hands-on, it is clear that Spring Data JPA allows developers to perform database operations with minimal code while still benefiting from Hibernate's powerful ORM capabilities. This makes Spring Data JPA the preferred choice for modern Spring Boot applications.

Screenshots:

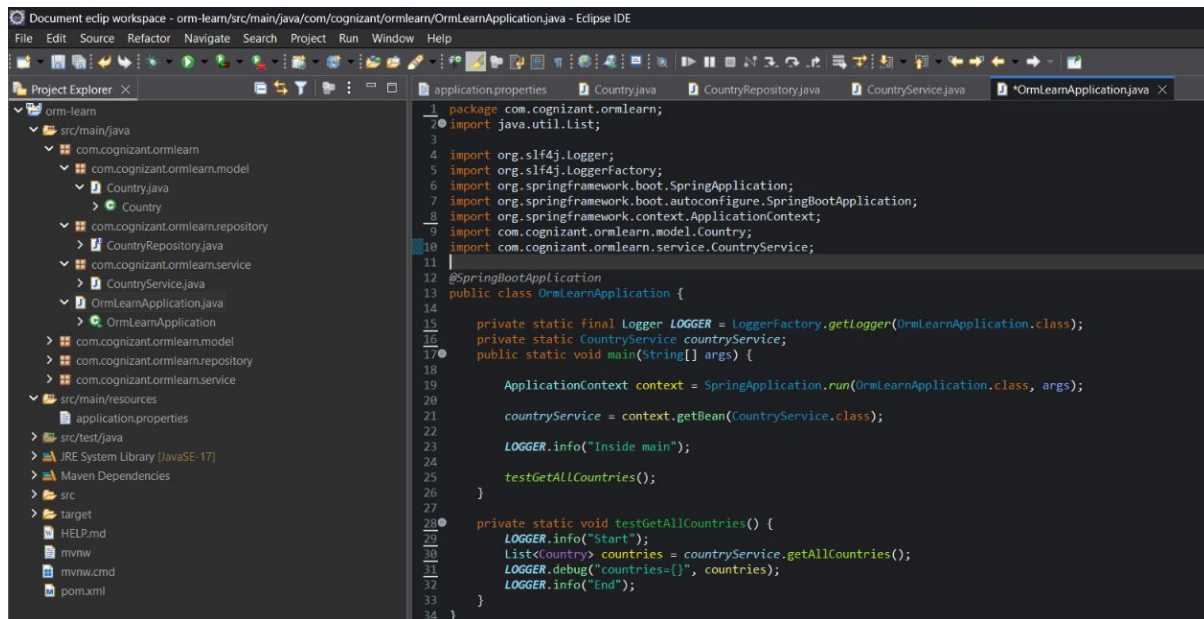
CountryRepository.java



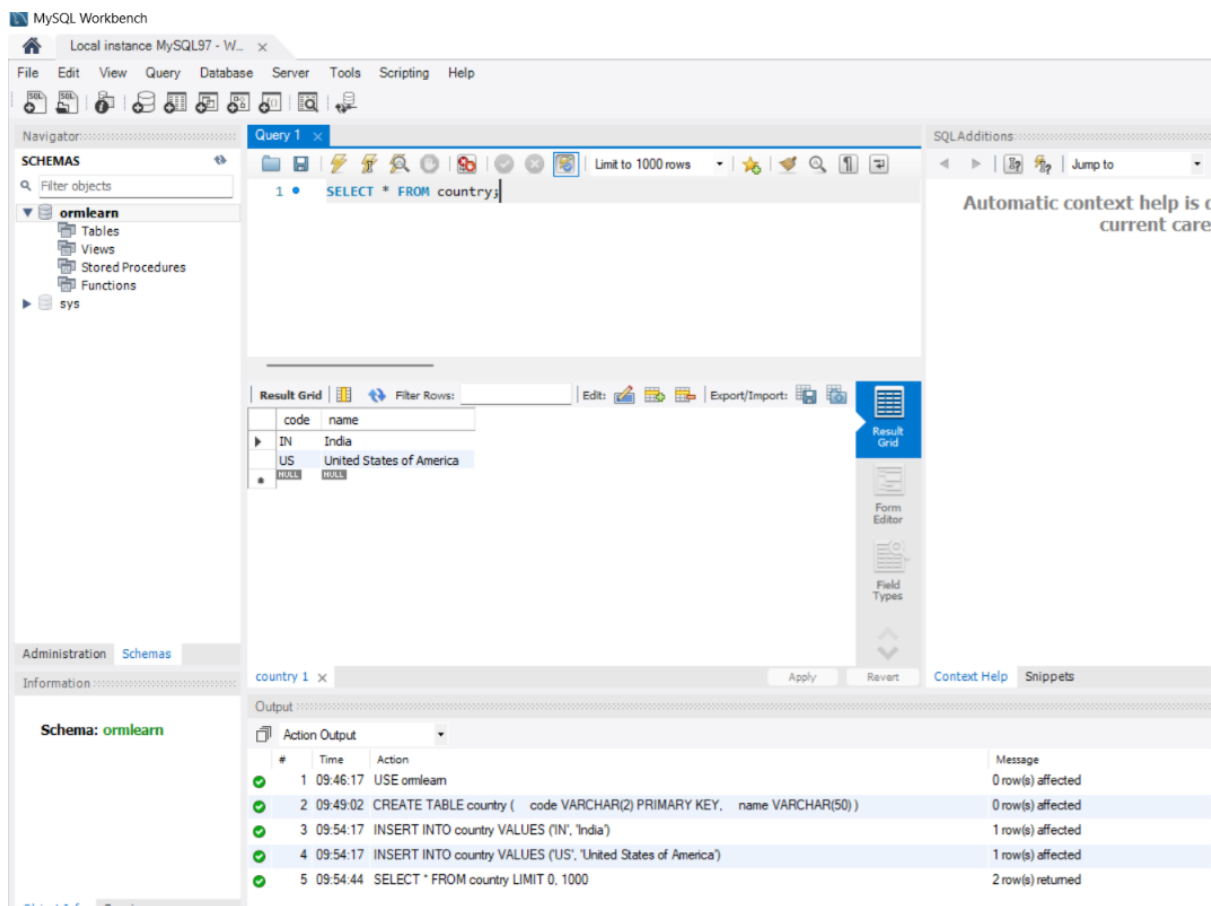
CountryService.java



OrmLearnApplication.java



```
1 package com.cognizant.ormlearn;
2 import java.util.List;
3
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.boot.SpringApplication;
7 import org.springframework.boot.autoconfigure.SpringBootApplication;
8 import org.springframework.context.ApplicationContext;
9 import com.cognizant.ormlearn.model.Country;
10 import com.cognizant.ormlearn.service.CountryService;
11
12 @SpringBootApplication
13 public class OrmLearnApplication {
14
15     private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);
16     private static CountryService countryService;
17     public static void main(String[] args) {
18
19         ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
20         countryService = context.getBean(CountryService.class);
21
22         LOGGER.info("Inside main");
23         testGetAllCountries();
24     }
25
26     private static void testGetAllCountries() {
27         LOGGER.info("Start");
28         List<Country> countries = countryService.getAllCountries();
29         LOGGER.debug("countries={}", countries);
30         LOGGER.info("End");
31     }
32 }
```



MySQL Workbench

Local instance MySQL97 - W...

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

ormlearn

- Tables
- Views
- Stored Procedures
- Functions

sys

Query 1

1 • SELECT * FROM country;

Limit to 1000 rows

Result Grid

code	name
IN	India
US	United States of America

country 1 x

Apply Revert Context Help Snippets

Output

#	Time	Action	Message
1	09:46:17	USE ormlearn	0 row(s) affected
2	09:49:02	CREATE TABLE country (code VARCHAR(2) PRIMARY KEY, name VARCHAR(50))	0 row(s) affected
3	09:54:17	INSERT INTO country VALUES ('IN', 'India')	1 row(s) affected
4	09:54:17	INSERT INTO country VALUES ('US', 'United States of America')	1 row(s) affected
5	09:54:44	SELECT * FROM country LIMIT 0, 1000	2 row(s) returned

